

SIGCB-QR

Sistema de Gestion de Biblioteca con Codigos QR

ENTREGA - TERCERA ENTREGA

v0.9.0-rc — Julio 2026

Documento consolidado: SRS, Casos de Uso, Historias de Usuario,
Matriz de Trazabilidad,Codigo Fuente y Apendices

Tabla de Contenidos

Portada

SRS — Documento de Requisitos

Software Requirements Specification (SRS)

SIGCB-QR -- Sistema Integral de Gestion Bibliotecaria

Version:
v0.9.0-rc

Fecha:
2026-07-30

Conforme a:
ISO/IEC/IEEE 29148:2018

1. Introduccion

1.1 Proposito

Este documento especifica los requisitos funcionales y no funcionales del Sistema Integral de Gestion, Control y Seguimiento de Lectura Interna en la Biblioteca Universitaria mediante Codigos QR (SIGCB-QR). El sistema automatiza los procesos bibliotecarios: autenticacion, gestion de catalogo, prestamos, reservas, multas y reportes.

1.2 Alcance

SIGCB-QR es una aplicacion web de arquitectura cliente-servidor con backend Spring Boot 3.4, frontend Laravel 13, base de datos PostgreSQL 16 y cache Redis 7. El sistema expone una API REST documentada con OpenAPI 3.0 y utiliza autenticacion stateless JWT en cookie HttpOnly.

1.3 Definiciones

Termino	Definicion
JWT	JSON Web Token (RFC 7519)
JPA	Jakarta Persistence API
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
TTL	Time To Live
Hit ratio	Proporcion de aciertos de cache

1.4 Referencias

- ISO/IEC/IEEE 29148:2018 -- Ingenieria de requisitos
- RFC 7519 -- JSON Web Token
- RFC 7807 -- Problem Details for HTTP APIs
- INCOSE Guide to Writing Requirements v4
- OWASP Cheat Sheet -- SQL Injection Prevention
- Cockburn -- Writing Effective Use Cases

1.5 Resumen

El SRS se organiza en: descripcion global (seccion 2), requisitos especificos funcionales (3), no funcionales (4), de interfaz externa (5), de seguridad (6) y de calidad (7).

2. Descripcion global

2.1 Perspectiva del producto

SIGCB-QR es un sistema independiente que reemplaza procesos manuales bibliotecarios. Se compone de:

- API REST (sigcb-qr-api): Java 21, Spring Boot 3.4.4
- Frontend (SIGCB-QR): Laravel 13, PHP 8.4
- Base de datos: PostgreSQL 16
- Cache: Redis 7
- Contenedores: Docker Compose

2.2 Funciones del sistema

- Autenticacion stateless con JWT (login, registro, logout)
- CRUD de usuarios (admin)
- CRUD de material bibliografico (libros, autores, editoriales, categorias)
- Gestion de ejemplares (inventario)
- Prestamos, devoluciones y renovaciones
- Reservas
- Multas y sanciones
- Reportes y estadisticas de dashboard
- Catalogacion (facultades, carreras)

2.3 Caracteristicas de usuarios

RoL	Descripcion
ADMIN	Administracion completa del sistema
BIBLIOTECARIO	Gestion de prestamos, catalogo y reportes
ESTUDIANTE	Consulta de catalogo, prestamos y reservas

2.4 Restricciones

- Autenticacion stateless (sin sesiones HTTP)
- JWT transmitido en cookie HttpOnly + Secure + SameSite=Strict
- Base de datos gestionada exclusivamente por Flyway (ddl-auto=validate)
- Operaciones no elementales mediante stored procedures
- Contenedores Docker anclados por digest SHA256

2.5 Supuestos y dependencias

- Disponibilidad de Docker y Docker Compose
- Conexion de red entre contenedores
- Semillas deterministas fijadas

3. Requisitos especificos -- Funcionales

REQ-F-001 -- Autenticacion de usuarios

Atributo	Valor
Tipo	Funcional
Prioridad (MoSCoW)	Must
Enunciado	Al recibir credenciales validas, el sistema debera emitir un JWT firmado en cook
Rationale	La autenticacion stateless elimina la necesidad de almacenar sesiones en servido
Origen	Stakeholder: Administracion bibliotecaria
Criterio de aceptacion	Login exitoso en menos de 2s con JWT valido
Metodo de verificacion	Test (AuthControllerTest)
Trazabilidad	HU-01, CU-01

REQ-F-002 -- Registro de usuarios

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	Al recibir datos de registro validos, el sistema debera crear un usuario con rol
Rationale	Permitir auto-registro de estudiantes
Criterio de aceptacion	Registro exitoso con validacion de email unico
Metodo de verificacion	Test
Trazabilidad	HU-02, CU-01

REQ-F-003 -- Cierre de sesion

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	Al recibir una solicitud de logout, el sistema debera agregar el JTI del token a
Rationale	Garantizar que el token no pueda reutilizarse
Criterio de aceptacion	Token blacklistado no es valido para peticiones posteriores
Metodo de verificacion	Test
Trazabilidad	HU-03, CU-01

REQ-F-004 -- CRUD de usuarios (Admin)

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	El sistema debera permitir al administrador crear, leer, actualizar y desactivar
Rationale	Gestion centralizada de cuentas
Criterio de aceptacion	CRUD completo con validaciones
Metodo de verificacion	Test
Trazabilidad	HU-04

REQ-F-005 -- CRUD de material bibliografico

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	El sistema debera permitir crear, leer, actualizar y desactivar libros con sus m
Rationale	Catalogacion del acervo bibliotecario
Criterio de aceptacion	CRUD completo con paginacion y busqueda
Metodo de verificacion	Test
Trazabilidad	HU-05, CU-02

REQ-F-006 -- Prestamo de ejemplares

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	Al recibir una solicitud de prestamo valida, el sistema debera crear el prestamo
Rationale	Control de circulacion de material
Criterio de aceptacion	Prestamo creado en menos de 2s, maximo 5 activos por usuario
Metodo de verificacion	Test + stored procedure
Trazabilidad	HU-06, CU-03

REQ-F-007 -- Devolucion de prestamo

Atributo	Valor
Tipo	Funcional
Prioridad	Must
Enunciado	Al recibir una devolucion, el sistema debera actualizar el estado del prestamo,
Rationale	Control de inventario y penalizaciones
Criterio de aceptacion	Devolucion procesada en menos de 2s
Metodo de verificacion	Test + stored procedure
Trazabilidad	HU-07, CU-04

REQ-F-008 -- Reporte de prestamos diarios

Atributo	Valor
Tipo	Funcional
Prioridad	Should
Enunciado	El sistema debera generar un reporte de prestamos del dia actual con datos de us
Rationale	Seguimiento de actividad diaria
Criterio de aceptacion	Reporte generado en menos de 3s
Metodo de verificacion	Test + stored procedure

Trazabilidad	HU-08
--------------	-------

4. Requisitos no funcionales

REQ-NF-001 -- Rendimiento -- Listado de libros

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	El endpoint GET /api/libros debera responder con p95 <= 200 ms con cache Redis c
Rationale	Experiencia de usuario fluida en navegacion de catalogo
Criterio de aceptacion	p95 < 200ms en 3 corridas con cache caliente
Metodo de verificacion	Test de rendimiento (k6)
Trazabilidad	Evidencia: docs/mediciones/perf/

REQ-NF-002 -- Seguridad -- Autenticacion

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	El sistema debera implementar autenticacion JWT sin sesiones, con cookie HttpOnl
Rationale	Mitigacion de ataques XSS y CSRF
Criterio de aceptacion	Ningun endpoint protegido accesible sin JWT valido
Metodo de verificacion	Test de seguridad
Trazabilidad	OWASP Top 10

REQ-NF-003 -- Seguridad -- Proteccion contra inyeccion SQL

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	Ninguna consulta SQL debera concatenar entrada de usuario; toda parametrizacion
Rationale	Prevencion de inyeccion SQL
Criterio de aceptacion	Escaneo SAST sin hallazgos de inyeccion SQL
Metodo de verificacion	Analisis estatico (inspection)
Trazabilidad	OWASP Cheat Sheet

REQ-NF-004 -- Disponibilidad -- Contenedores

Atributo	Valor
Tipo	No funcional
Prioridad	Must
Enunciado	El sistema debera poder reconstruirse en menos de 2 minutos desde una clonacion
Rationale	Reproducibilidad por terceros
Criterio de aceptacion	docker-compose up exitoso en ambiente limpio

Metodo de verificacion	Demostracion (demonstration)
Trazabilidad	Docker Compose

REQ-NF-005 -- Mantenibilidad -- Cobertura de pruebas

Atributo	Valor
Tipo	No funcional
Prioridad	Should
Enunciado	El backend debera tener cobertura de linea >= 30% medida por JaCoCo
Rationale	Garantia minima de calidad
Criterio de aceptacion	Reporte JaCoCo >= 30%
Metodo de verificacion	Test (JaCoCo)
Trazabilidad	pom.xml

5. Requisitos de interfaz externa

5.1 Interfaz de usuario

Frontend Laravel 13 con Tailwind CSS, accesible via navegador en puerto 8000.

5.2 Interfaz API REST

Documentacion OpenAPI 3.0 en /api-docs e interfaz Swagger UI en /swagger-ui.html .

5.3 Interfaz de base de datos

Conexion JDBC a PostgreSQL 16, esquema gestionado por Flyway.

6. Requisitos de seguridad

- JWT con claims estandar RFC 7519
- Cookie HttpOnly + Secure + SameSite=Strict
- Blacklist de JTI revocados en Redis
- CORS restringido a origenes configurados
- BCrypt para almacenamiento de contraseñas
- Parametros nombrados en todas las consultas
- Procedimientos almacenados sin SQL dinamico

7. Calidad de software

7.1 Caracteristicas del conjunto de requisitos (INCOSE C10-C15)

Atributo	Cumplimiento
Complete (C10)	Todos los escenarios principales cubiertos
Consistent (C11)	Sin contradicciones entre requisitos
Feasible (C12)	Implementable con la pila tecnologica actual
Comprehensible (C13)	Redactado en lenguaje claro
Able to be validated (C14)	Cada requisito tiene metodo de verificacion
Correct (C15)	Aprobado por stakeholders

Fin del SRS

CU-01: Caso de uso

CU-01: Autenticación de usuarios

Nivel:

1 (Resumen)

Actor principal:

Usuario no autenticado

Stakeholders:

Administración bibliotecaria (desea control de acceso)

Escenario principal de éxito

- El usuario envía sus credenciales (email, contraseña) al sistema
- El sistema valida las credenciales contra la base de datos
- El sistema genera un JWT firmado con claims estándar (iss, sub, aud, exp, nbf, iat, jti)
- El sistema establece una cookie HttpOnly + Secure + SameSite=Strict con el JWT
- El usuario queda autenticado y puede acceder a recursos protegidos
- El sistema permite al usuario cerrar sesión, agregando el JTI a la blacklist

Extensiones

1a. Credenciales inválidas

1a1. El sistema responde con HTTP 401 y ProblemDetail

2a. Token expirado

2a1. El sistema rechaza la petición con HTTP 401

3a. Token blacklistado

3a1. El sistema rechaza la petición con HTTP 401

Requisitos asociados

- REQ-F-001 (Login)
- REQ-F-002 (Registro)
- REQ-F-003 (Logout)
- REQ-NF-002 (Seguridad)

CU-02: Caso de uso

CU-02: Gestión de material bibliográfico

Nivel:

2 (Etapa de objetivo de usuario)

Actor principal:

Bibliotecario

Precondición:

Autenticado con rol BIBLIOTECARIO o ADMIN

Escenario principal de éxito

- El bibliotecario solicita el listado de libros (paginado)
- El sistema retorna los libros activos con paginación, desde caché Redis si está caliente
- El bibliotecario busca un libro por título
- El sistema retorna resultados de búsqueda
- El bibliotecario crea un nuevo libro con título, ISBN, editorial, categoría y autores
- El sistema valida los datos y crea el libro con ejemplares disponibles = totales
- El bibliotecario actualiza datos de un libro existente
- El bibliotecario desactiva un libro (eliminación lógica)

Extensiones

5a. Datos inválidos

5a1. El sistema responde con HTTP 400 y errores de validación

7a. Libro no encontrado

7a1. El sistema responde con HTTP 404 y ProblemDetail

Requisitos asociados

- REQ-F-005 (CRUD libros)
- REQ-NF-001 (Rendimiento con caché)

CU-03: Caso de uso

CU-03: Prestamo de ejemplares

Nivel:

2 (Etapa de objetivo de usuario)

Actor principal:

Bibliotecario

Precondicion:

Autenticado con rol BIBLIOTECARIO o ADMIN

Postcondicion:

Prestamo registrado, ejemplar marcado como PRESTADO

Escenario principal de exito

- El bibliotecario selecciona un usuario del sistema
- El bibliotecario selecciona un ejemplar disponible del inventario
- El sistema valida que el usuario no tenga mas de 5 prestamos activos
- El sistema valida que el ejemplar este DISPONIBLE
- El sistema crea el registro de prestamo con fecha actual y vencimiento en 7 dias
- El sistema cambia el estado del ejemplar a PRESTADO
- El sistema disminuye el contador de ejemplares disponibles del libro
- El sistema retorna el detalle del prestamo creado

Extensiones

3a. Limite de prestamos alcanzado

3a1. El sistema rechaza la operacion con HTTP 400

3a2. Mensaje: "El usuario tiene demasiados prestamos activos"

4a. Ejemplar no disponible

4a1. El sistema rechaza la operacion con HTTP 400

4a2. Mensaje: "El ejemplar no esta disponible"

5a. Error de base de datos

5a1. El sistema revierte la transaccion

5a2. El sistema retorna HTTP 500 con ProblemDetail

Requisitos asociados

- REQ-F-006 (Prestamo de ejemplares)
- REQ-NF-006 (Control de acceso por roles)
- REQ-R-003 (Tiempo de respuesta de SP)

CU-04: Caso de uso

CU-04: Devolucion de prestamo

Nivel:

2 (Etapa de objetivo de usuario)

Actor principal:

Bibliotecario

Precondicion:

Autenticado con rol BIBLIOTECARIO o ADMIN

Postcondicion:

Prestamo cerrado, inventario actualizado, multa generada si aplica

Escenario principal de exito

- El bibliotecario localiza el prestamo activo por su ID
- El sistema verifica que el prestamo exista y este ACTIVO
- El sistema registra la fecha de devolucion (fecha actual)
- El sistema cambia el estado del prestamo a DEVUELTO
- El sistema cambia el estado del ejemplar a DISPONIBLE
- El sistema incrementa el contador de ejemplares disponibles del libro
- Si la fecha actual supera la fecha de vencimiento, el sistema genera una multa
- El sistema retorna el detalle del prestamo actualizado

Extensiones

2a. Prestamo no encontrado

2a1. El sistema retorna HTTP 404 con ProblemDetail

2b. Prestamo ya devuelto

2b1. El sistema rechaza con HTTP 400

2b2. Mensaje: "El prestamo ya fue devuelto"

7a. Calculo de multa

7a1. $\text{Dias de retraso} = \text{fecha_actual} - \text{fecha_vencimiento}$

7a2. $\text{Monto} = \text{dias_retraso} * \0.50

7a3. La multa se registra con estado PENDIENTE

Requisitos asociados

- REQ-F-007 (Devolucion de prestamo)
- REQ-R-003 (Tiempo de respuesta de SP)

CU-05: Caso de uso

CU-05: Generacion de reportes

Nivel:

1 (Resumen)

Actor principal:

Bibliotecario/Administrador

Precondicion:

Autenticado con rol BIBLIOTECARIO o ADMIN

Postcondicion:

Reporte generado y entregado al solicitante

Escenario principal de exito

- El usuario solicita un tipo de reporte (prestamos diarios, libros mas solicitados, multas cobradas)
- El sistema ejecuta el stored procedure correspondiente
- El sistema recopila los datos agregados
- El sistema retorna los datos en formato JSON

Extensiones

2a. Sin datos para el periodo

2a1. El sistema retorna un reporte vacio con total = 0

3a. Error de base de datos

3a1. El sistema retorna HTTP 500 con ProblemDetail

Requisitos asociados

- REQ-F-009 (Reporte de prestamos diarios)
- REQ-NF-006 (Control de acceso por roles)

HU-01: Historia de usuario

HU-01: Autenticación de usuarios

Formato:
Connextra

Rol:
Usuario del sistema

Objetivo:

Iniciar sesión con credenciales

Beneficio:
Acceder a las funcionalidades del sistema según mi rol

Criteria INVEST:
Independent Negotiable Valuable Estimable Small Testable

Criterios de aceptación (Gherkin)

[illegible]

HU-02: Historia de usuario

HU-02: Registro de usuarios

Formato:

Connextra

Rol:

Visitante

Objetivo:

Registrarme como usuario del sistema

Beneficio:

Obtener acceso como estudiante a los servicios bibliotecarios

Criterios de aceptación (Gherkin)

```

Registeo
Regístrese
Conéctese
Registrar
Iniciar sesión
REGISTRARSE
AUTOR
8
Registado
Conéctese@test.com
Iniciar sesión
Iniciar sesión
Iniciar sesión

```


HU-06: Historia de usuario

HU-06: Prestamo de ejemplares

Formato:
Connextra

Rol:
Bibliotecario

Objetivo:
Registrar el prestamo de un ejemplar a un usuario

Beneficio:
Controlar la circulacion del material bibliotecario

Criterios de aceptacion (Gherkin)

Escenarios
Escenario
Escenario
Bibliotecario
Préstamo
Escenario
Escenarios
8
Escenario
Escenario
Bibliotecario
Préstamo
Escenario
14
Escenario
Escenarios
Bibliotecario
Préstamo
Préstamo

HU-09: Historia de usuario

HU-09: Reportes bibliotecarios

Formato:
Connextra

Rol:
Bibliotecario/Administrador

Objetivo:
Generar reportes de actividad

Beneficio:
Obtener datos para la toma de decisiones

Criterios de aceptacion (Gherkin)

Reportes
Reportes
Bibliotecario
Reportes
Reportes
Reportes
7
Reportes
Bibliotecario
Reportes
Reportes
12
Reportes
Administrador
Reportes
Reportes

HU-10: Historia de usuario

HU-10: Dashboard de estadísticas

Formato:
Connextra

Rol:
Usuario autenticado

Objetivo:
Visualizar estadísticas generales del sistema

Beneficio:
Obtener una vision rapida del estado de la biblioteca

Criterios de aceptacion (Gherkin)

Dashboard
Estadísticas
Autenticado
Problemas
Posibles
Admisores
Admisores
Admisores
Admisores

Matriz de Trazabilidad

id_req	tipo	prioridad_m	historia_u	caso_de	modulo_codigo	endpoint_api	prueba_automatiz	tipo_ac	evidencia_empiri	estado	
REQ-F-00	Funcional	Must	HU-01	CU-01	AuthController	POST /api/auth/log	AuthControllerTest	CRUD-ORM		verificado	
REQ-F-00	Funcional	Must	HU-01	CU-01	JwtTokenProvider		JwtTokenProviderTe	CRUD-ORM		verificado	
REQ-F-00	Funcional	Must	HU-02	CU-01	AuthController	POST /api/auth/reg	AuthControllerTest	CRUD-ORM		verificado	
REQ-F-00	Funcional	Must	HU-03	CU-01	AuthController	POST /api/auth/log	AuthControllerTest	CRUD-ORM		verificado	
REQ-F-00	Funcional	Must	HU-04		UsuarioController	GET/POST/PUT/DELET	UsuarioControllerT	CRUD-ORM		verificado	
REQ-F-00	Funcional	Must	HU-05	CU-02	LibroController	GET/POST/PUT/DELET	LibroControllerTes	CRUD-ORM		verificado	
REQ-F-00	Funcional	Must	HU-06	CU-03	PrestamoControlle	POST /api/prestamo	PrestamoServiceTes	SP	docs/mediciones/pe	verificado	
REQ-F-00	Funcional	Must	HU-07	CU-04	PrestamoControlle	PUT /api/prestamos	PrestamoServiceTes	SP		verificado	
REQ-F-00	Funcional	Should	HU-08		PrestamoControlle	PUT /api/prestamos		SP		implementado	
REQ-F-00	Funcional	Should	HU-09	CU-05	ReporteController	GET /api/reportes/	ReporteControllerT	SP		implementado	
REQ-F-01	Funcional	Must	HU-10		DashboardControlle	GET /api/dashboard		SP		verificado	
REQ-F-01	Funcional	Should			ReservaController	POST/DELETE /api/r		CRUD-ORM		implementado	
REQ-NF-0	No funcio	Must	HU-05		LibroService	GET /api/libros	LibroPerfTest.p95_	CRUD-ORM	docs/mediciones/pe	verificado	
REQ-NF-0	No funcio	Must	HU-01	CU-01	SecurityConfig		SecurityTest.endpo	CRUD-ORM		verificado	
REQ-NF-0	No funcio	Must			GlobalExceptionHa		SecurityTest.sql_i	CRUD-ORM		verificado	
REQ-NF-0	No funcio	Must				docker-compose.yml				docs/mediciones/pe	pendiente
REQ-NF-0	No funcio	Should				pom.xml		JaCoCo		verificado	
REQ-NF-0	No funcio	Must	HU-04	CU-01	SecurityConfig	@PreAuthorize	ReporteControllerT	CRUD-ORM		verificado	
REQ-NF-0	No funcio	Must	HU-03		JwtBlacklistServ		JwtBlacklistServic	CRUD-ORM		verificado	
REQ-R-00	Rendimier	Must			AuthController	POST /api/auth/log		CRUD-ORM	docs/mediciones/pe	pendiente	
REQ-R-00	Rendimier	Must			LibroController	GET/POST/PUT/DELET		CRUD-ORM	docs/mediciones/pe	pendiente	
REQ-R-00	Rendimier	Should			PrestamoControlle	POST /api/prestamo		SP	docs/mediciones/pe	pendiente	
REQ-R-00	Rendimier	Should			LibroService	GET /api/libros		CRUD-ORM	docs/mediciones/pe	pendiente	

Bitacora de Observaciones

Bitácora de Observaciones — SIGCB-QR

Archivo de trazabilidad de observaciones recibidas del docente en las Entregas 1A y 1B,
conforme al requisito Bloque 0
de la Tercera Entrega (v0.9.0-rc).

Resumen

Indicador	Valor
Total observaciones	7
Resueltas	7
No resueltas	0
% Resuelto	100 %

Observaciones

El JWT no incluye el claim aud Aplicada: se agregaron los claims iss , aud , nbf e iat La cookie JWT tiene Secure=false Apl.

Historial de cambios

Fecha	Autor	Cambio
2026-07-30	Equipo SIGCB-QR	Creación inicial del archivo con 7 observaciones identificadas del an

CHANGELOG de Requisitos

Changelog de Requisitos — SIGCB-QR

Formato: Keep a Changelog adaptado a requisitos.[v0.9.0-rc] — 2026-07-30

Added

- REQ-F-008: Renovacion de prestamo
- REQ-F-009: Reporte de prestamos diarios
- REQ-F-010: Dashboard de estadisticas
- REQ-F-011: Gestion de reservas
- REQ-NF-005: Cobertura de pruebas $\geq 30\%$ (JaCoCo)
- REQ-NF-006: Control de acceso por roles (RBAC)
- REQ-NF-007: Cache Redis para blacklist JWT
- REQ-R-001: Tiempo de respuesta de autenticacion (p95 $\leq 500\text{ms}$)
- REQ-R-002: Tiempo de respuesta de CRUD (p95 $\leq 300\text{ms}$)
- REQ-R-003: Tiempo de respuesta de stored procedures ($< 1\text{s}$)
- REQ-R-004: Hit ratio de cache Redis $\geq 80\%$
- HU-06: Prestamo de ejemplares
- HU-07: Devolucion de prestamo
- HU-08: Renovacion de prestamo
- HU-09: Reportes bibliotecarios
- HU-10: Dashboard de estadisticas
- CU-03: Prestamo de ejemplares
- CU-04: Devolucion de prestamo
- CU-05: Generacion de reportes
- SRS estructurado completo segun ISO/IEC/IEEE 29148:2018
- Matriz de trazabilidad con 23 filas
- SRS.pdf generado

Modified

- REQ-F-001: Claims JWT ampliados (iss, aud, nbf, iat agregados)
- REQ-F-001: Cookie JWT ahora usa Secure desde configuracion externa
- REQ-F-003: Logout ahora usa blacklist JWT
- REQ-F-005: Cache Redis agregado con @Cacheable
- REQ-F-006: Migrado a stored procedure (sp_crear_prestamo)
- REQ-F-007: Migrado a stored procedure (sp_devolver_prestamo)
- REQ-NF-002: Endurecido con SameSite=Strict + Secure configurable
- Error handling: migrado de ApiResponse a RFC 7807 ProblemDetails

[v0.7.0] — 2026-07-14 (Entrega 1B)

Added

- REQ-F-001: Autenticacion stateless JWT
- REQ-F-002: Registro de usuarios
- REQ-F-003: Cierre de sesion con JTI blacklist
- REQ-F-004: CRUD de usuarios (Admin)
- REQ-F-005: CRUD de material bibliografico
- REQ-F-006: Prestamo de ejemplares (via JPA)
- REQ-F-007: Devolucion de prestamo (via JPA)

- REQ-NF-001: Rendimiento de listado de libros
- REQ-NF-002: Seguridad de autenticacion
- REQ-NF-003: Proteccion contra inyeccion SQL
- REQ-NF-004: Disponibilidad mediante Docker Compose

[v0.3.0] — 2026-06-04 (Entrega 1A)

Added

- SRS inicial con requisitos funcionales y no funcionales base
- Casos de uso y arquitectura C4 nivel 1 y 2

Catalogo de Stored Procedures

Catálogo de Procedimientos Almacenados y Funciones — SIGCB-QR

Versión: v0.9.0-rc

Fecha: 2026-07-30

Convención de nomenclatura

fn_ db/procs/fn_<verbo>_<sustantivo>.sql	sp_ db/procs/sp_<verbo>_<sustantivo>.sql	Prefijo	Tipo
			Función
			Procedimiento

Funciones

fn_contar_prestamos_entre_fechas

p_inicio TIMESTAMP , p_fin TIMESTAMP BIGINT prestamos	Propiedad	Valor
	Propósito	Cuenta préstamos
	Parámetros	
	Retorno	
	Tablas afectadas	(SELECT)

fn_total_multas_pendientes

DECIMAL(12,2) multas	Propiedad	Valor
	Propósito	Suma el monto total de multas no pagadas
	Parámetros	Ninguno
	Retorno	
	Tablas afectadas	(SELECT)

fn_total_cobrado_entre

p_inicio TIMESTAMP , p_fin TIMESTAMP DECIMAL(12,2) multas	Propiedad	Valor
	Propósito	Suma el monto
	Parámetros	
	Retorno	
	Tablas afectadas	(SELECT)

fn_libros_disponibles

TABLE(id BIGINT, titulo VARCHAR, isbn VARCHAR, ejemplares_disponibles INT) libros	Propiedad	
	Propósito	
	Parámetros	
	Retorno	
	Tablas afectadas	

Procedimientos almacenados

sp_reporte_libros_mas_prestados

p_limit INT (IN), p_ref REFCURSOR libros

Propiedad	Valor
Propósito	Obtiene los libros más prestados
Parámetros	(INOUT)
Cursores	Retorna cursor con id, titulo, i
Tablas afectadas	(SELECT)

sp_prestamos_entre_fechas

p_inicio TIMESTAMP (IN), p_fin TIMESTAMP (IN), p_ref REFCURSOR prestamos , usuarios , inventario , libros

Propiedad
Propósito
Parámetros
Cursores
Tablas afectadas

sp_top_usuarios_prestamos

p_limit INT (IN), p_ref REFCURSOR usuarios , prestamos

Propiedad	Valor
Propósito	Obtiene los u
Parámetros	(INOUT)
Cursores	Retorna curso
Tablas afectadas	(SELECT, JOIN

sp_dashboard_estadisticas

p_fecha_inicio TIMESTAMP (IN), p_fecha_fin TIMESTAMP (IN), p_ref REFCURSOR prestamos , libros , usuarios , reservas , multa

sp_crear_prestamo

p_usuario_id BIGINT (IN), p_inventario_id BIGINT (IN), p_dias_prestamo INT DEFAULT 7 (IN), p_prestamo_id BIGINT DEFAULT NULL

sp_devolver_prestamo

p_prestamo_id BIGINT (IN), p_multa_generada BOOLEAN DEFAULT FALSE prestamos (UPDATE), inventario (UPDATE), libros (UPDATE),

sp_renovar_prestamo

p_prestamo_id BIGINT (IN), p_dias Renovacion INT DEFAULT 7 (IN), p_nuevo_prestamo_id BIGINT DEFAULT NULL prestamos

Propiedad
Propósito
Parámetros
Validaciones
Tablas afectadas

sp_reporte_prestamos_diarios

p_fecha DATE (IN), p_ref REFCURSOR prestamos , usuarios , inventario , libros

Propiedad
Propósito
Parámetros
Cursores
Tablas afectadas

sp_reporte_multas_cobradas

p_mes INT (IN), p_anio INT (IN), p_ref REFCURSOR multas

Propiedad	Valor
Propósito	Genera report
Parámetros	(INOUT)
Cursores	Retorna curs
Tablas afectadas	(SELECT, CO

Seguridad

Todos los procedimientos usan exclusivamente parámetros nombrados. No se construye SQL dinámico mediante concatenación (EXECUTE IMMEDIATE, sp_executesql o equivalentes).

La cuenta de base de datos de la aplicación tiene únicamente:- EXECUTE sobre estos procedimientos y funciones- SELECT , I

sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtTokenProvider.java

```

package com.sigcbqr.security;
2
import io.jsonwebtoken.*;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
10
import javax.crypto.SecretKey;
import java.util.Date;
import java.util.UUID;
14
@Component
public class JwtTokenProvider {
17
18     private final SecretKey secretKey;
19     private final long expirationMs;
20     private final String issuer;
21     private final String audience;
22     private final boolean secureCookie;
23     private final JwtBlacklistService blacklistService;
24
25     public JwtTokenProvider(
26         @Value("${app.jwt.secret}") String secret,
27         @Value("${app.jwt.expiration-ms}") long expirationMs,
28         @Value("${app.jwt.issuer}") String issuer,
29         @Value("${app.jwt.audience}") String audience,
30         @Value("${app.jwt.secure-cookie}") boolean secureCookie,
31         JwtBlacklistService blacklistService) {
32         this.secretKey = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
33         this.expirationMs = expirationMs;
34         this.issuer = issuer;
35         this.audience = audience;
36         this.secureCookie = secureCookie;
37         this.blacklistService = blacklistService;
38     }
39
40     public String generateToken(Long userId, String email, String rol) {
41         Date now = new Date();
42         Date expiryDate = new Date(now.getTime() + expirationMs);
43
44         return Jwts.builder()
45             .id(UUID.randomUUID().toString())
46             .issuer(issuer)
47             .subject(userId.toString())
48             .audience().add(audience).and()
49             .claim("email", email)
50             .claim("rol", rol)
51             .issuedAt(now)
52             .notBefore(now)
53             .expiration(expiryDate)
54             .signWith(secretKey)
55             .compact();
56     }
57
58     public Long getUserIdFromToken(String token) {
59         Claims claims = parseToken(token);
60         return Long.parseLong(claims.getSubject());
61     }
62
63     public String getEmailFromToken(String token) {
64         Claims claims = parseToken(token);
65         return claims.get("email", String.class);
66     }
67
68     public String getRolFromToken(String token) {
69         Claims claims = parseToken(token);
70         return claims.get("rol", String.class);
71     }
72
73     public String getJtiFromToken(String token) {
74         Claims claims = parseToken(token);
75         return claims.getId();
76     }
77
78     public boolean validateToken(String token) {
79         try {
80             Claims claims = parseToken(token);
81             if (blacklistService.isBlacklisted(claims.getId())) {

```


sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtTokenProvider.java (cont.)

```

82         return false;
83     }
84     return true;
85 } catch (JwtException | IllegalArgumentException e) {
86     return false;
87 }
88 }
89
90 private Claims parseToken(String token) {
91     return Jwts.parser()
92         .verifyWith(secretKey)
93         .build()
94         .parseSignedClaims(token)
95         .getPayload();
96 }
97
98 public String extractTokenFromCookie(HttpServletRequest request) {
99     Cookie[] cookies = request.getCookies();
100     if (cookies != null) {
101         for (Cookie cookie : cookies) {
102             if ("access_token".equals(cookie.getName())) {
103                 return cookie.getValue();
104             }
105         }
106     }
107     return null;
108 }
109
110 public Cookie createAccessTokenCookie(String token) {
111     Cookie cookie = new Cookie("access_token", token);
112     cookie.setHttpOnly(true);
113     cookie.setSecure(secureCookie);
114     cookie.setPath("/");
115     cookie.setMaxAge((int) (expirationMs / 1000));
116     cookie.setAttribute("SameSite", "Strict");
117     return cookie;
118 }
119
120 public Date getExpirationFromToken(String token) {
121     return parseToken(token).getExpiration();
122 }
123
124 public SecretKey getSecretKey() {
125     return secretKey;
126 }
127
128 public JwtBlacklistService getJwtBlacklistService() {
129     return blacklistService;
130 }
131
132 public Cookie createLogoutCookie() {
133     Cookie cookie = new Cookie("access_token", null);
134     cookie.setHttpOnly(true);
135     cookie.setSecure(secureCookie);
136     cookie.setPath("/");
137     cookie.setMaxAge(0);
138     cookie.setAttribute("SameSite", "Strict");
139     return cookie;
140 }
141 }

```

sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtAuthenticationFilter.java

```

package com.sigcbqr.security;
2
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;
14
import java.io.IOException;
import java.util.List;
17
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
20
    private final JwtTokenProvider tokenProvider;
22
    public JwtAuthenticationFilter(JwtTokenProvider tokenProvider) {
23
        this.tokenProvider = tokenProvider;
24
    }
25
    @Override
26
    protected void doFilterInternal(HttpServletRequest request,
27                                HttpServletResponse response,
28                                FilterChain filterChain) throws ServletException, IOException {
29
        String token = tokenProvider.extractTokenFromCookie(request);
30
        if (StringUtils.hasText(token) && tokenProvider.validateToken(token)) {
31
            Long userId = tokenProvider.getUserIdFromToken(token);
32
            String email = tokenProvider.getEmailFromToken(token);
33
            String rol = tokenProvider.getRolFromToken(token);
34
            List<SimpleGrantedAuthority> authorities = List.of(
35
                new SimpleGrantedAuthority("ROLE_" + rol)
36
            );
37
            UserPrincipal userPrincipal = new UserPrincipal(userId, email, null, rol, authorities);
38
            UsernamePasswordAuthenticationToken authentication =
39
                new UsernamePasswordAuthenticationToken(userPrincipal, null, authorities);
40
            authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
41
            SecurityContextHolder.getContext().setAuthentication(authentication);
42
        }
43
        filterChain.doFilter(request, response);
44
    }
45
}
46

```

sigcb-qr-api\src\main\java\com\sigcbqr\config\SecurityConfig.java

```

package com.sigcbqr.config;
2
import com.sigcbqr.security.JwtAuthenticationEntryPoint;
import com.sigcbqr.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
17
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
22
23     private final JwtAuthenticationFilter jwtAuthenticationFilter;
24     private final JwtAuthenticationEntryPoint jwtAuthenticationEntryPoint;
25
26     public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter,
27                          JwtAuthenticationEntryPoint jwtAuthenticationEntryPoint) {
28         this.jwtAuthenticationFilter = jwtAuthenticationFilter;
29         this.jwtAuthenticationEntryPoint = jwtAuthenticationEntryPoint;
30     }
31
32     @Bean
33     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
34         http
35             .csrf(csrf -> csrf.disable())
36             .cors(cors -> {})
37             .exceptionHandling(ex -> ex.authenticationEntryPoint(jwtAuthenticationEntryPoint))
38             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
39             .authorizeHttpRequests(auth -> auth
40                 .requestMatchers("/api/auth/**").permitAll()
41                 .requestMatchers("/swagger-ui/**", "/api-docs/**", "/v3/api-docs/**").permitAll()
42                 .requestMatchers("/api/dashboard/**").authenticated()
43                 .requestMatchers("/api/**").authenticated()
44                 .anyRequest().permitAll()
45             )
46             .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
47
48         return http.build();
49     }
50
51     @Bean
52     public PasswordEncoder passwordEncoder() {
53         return new BCryptPasswordEncoder();
54     }
55
56     @Bean
57     public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
58         return config.getAuthenticationManager();
59     }
60

```

sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtBlacklistService.java

```

package com.sigcbqr.security;
2
import com.sigcbqr.model.entity.JwtBlacklist;
import com.sigcbqr.repository.JwtBlacklistRepository;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
8
import java.time.LocalDateTime;
10
@Service
public class JwtBlacklistService {
13
    private final JwtBlacklistRepository repository;
15
    public JwtBlacklistService(JwtBlacklistRepository repository) {
16
        this.repository = repository;
17
    }
18
    @Transactional
19
    public void blacklist(String jti, LocalDateTime expiration) {
20
        if (!repository.existsByJti(jti)) {
21
            JwtBlacklist entry = JwtBlacklist.builder()
22
                .jti(jti)
23
                .fechaExpiracion(expiration)
24
                .build();
25
            repository.save(entry);
26
        }
27
    }
28
    public boolean isBlacklisted(String jti) {
29
        return repository.existsByJti(jti);
30
    }
31
    @Scheduled(cron = "0 0 */6 * * *")
32
    @Transactional
33
    public void cleanExpiredEntries() {
34
        repository.deleteByFechaExpiracionBefore(LocalDateTime.now());
35
    }
36
}
37

```

sigcb-qr-api\src\main\java\com\sigcbqr\controller\AuthController.java

```

package com.sigcbqr.controller;
2
import com.sigcbqr.model.dto.request.LoginRequest;
import com.sigcbqr.model.dto.request.RegisterRequest;
import com.sigcbqr.model.dto.response.ApiResponse;
import com.sigcbqr.model.dto.response.LoginResponse;
import com.sigcbqr.model.dto.response.UsuarioResponse;
import com.sigcbqr.security.JwtTokenProvider;
import com.sigcbqr.security.UserPrincipal;
import com.sigcbqr.service.AuthService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.*;
19
import java.time.LocalDateTime;
import java.util.Date;
22
@RestController
@RequestMapping("/api/auth")
@Tag(name = "Autenticación", description = "Endpoints de autenticación con JWT en cookie HttpOnly")
public class AuthController {
27
    private final AuthService authService;
    private final JwtTokenProvider tokenProvider;
30
    public AuthController(AuthService authService, JwtTokenProvider tokenProvider) {
32         this.authService = authService;
33         this.tokenProvider = tokenProvider;
34     }
35
36     @PostMapping("/login")
37     @Operation(summary = "Iniciar sesión", description = "Autentica al usuario y devuelve un JWT en cookie HttpOnly")
38     public ResponseEntity<ApiResponse> login(@Valid @RequestBody LoginRequest request,
39                                             HttpServletResponse response) {
40         LoginResponse loginResponse = authService.login(request);
41
42         String token = tokenProvider.generateToken(
43             loginResponse.getId(),
44             loginResponse.getEmail(),
45             loginResponse.getRol()
46         );
47
48         var cookie = tokenProvider.createAccessTokenCookie(token);
49         response.addCookie(cookie);
50
51         return ResponseEntity.ok(ApiResponse.success("Inicio de sesión exitoso", loginResponse));
52     }
53
54     @PostMapping("/register")
55     @Operation(summary = "Registrar usuario", description = "Registra un nuevo usuario y devuelve JWT en cookie")
56     public ResponseEntity<ApiResponse> register(@Valid @RequestBody RegisterRequest request,
57                                              HttpServletResponse response) {
58         LoginResponse loginResponse = authService.register(request);
59
60         String token = tokenProvider.generateToken(
61             loginResponse.getId(),
62             loginResponse.getEmail(),
63             loginResponse.getRol()
64         );
65
66         var cookie = tokenProvider.createAccessTokenCookie(token);
67         response.addCookie(cookie);
68
69         return ResponseEntity.ok(ApiResponse.success("Registro exitoso", loginResponse));
70     }
71
72     @PostMapping("/logout")
73     @Operation(summary = "Cerrar sesión", description = "Invalida el JWT actual agregando su JTI a la blacklist")
74     public ResponseEntity<ApiResponse> logout(HttpServletRequest request,
75                                             HttpServletResponse response) {
76         String token = tokenProvider.extractTokenFromCookie(request);
77         if (token != null && tokenProvider.validateToken(token)) {
78             Date expiration = tokenProvider.getExpirationFromToken(token);
79             authService.logout(token, new java.sql.Timestamp(expiration.getTime()).toLocalDateTime());
80         }
81

```

sigcb-qr-api\src\main\java\com\sigcbqr\controller\AuthController.java (cont.)

```
82     var cookie = tokenProvider.createLogoutCookie();
83     response.addCookie(cookie);
84
85     return ResponseEntity.ok(ApiResponse.success("Sesión cerrada exitosamente", null));
86 }
87
88 @GetMapping("/me")
89 @Operation(summary = "Usuario actual", description = "Obtiene los datos del usuario autenticado")
90 public ResponseEntity<ApiResponse> me(@AuthenticationPrincipal UserPrincipal userPrincipal) {
91     var usuario = authService.getCurrentUser(userPrincipal.id());
92     var response = UsuarioResponse.builder()
93         .id(usuario.getId())
94         .nombre(usuario.getNombre())
95         .email(usuario.getEmail())
96         .telefono(usuario.getTelefono())
97         .rol(usuario.getRol().getNombre())
98         .activo(usuario.getActivo())
99         .createdAt(usuario.getCreatedAt())
100        .build();
101     return ResponseEntity.ok(ApiResponse.success("Usuario actual", response));
102 }
103 }
```

sigcb-qr-api\src\main\java\com\sigcbqr\service\AuthService.java

```

package com.sigcbqr.service;
2
import com.sigcbqr.exception.BadRequestException;
import com.sigcbqr.exception.ResourceNotFoundException;
import com.sigcbqr.model.dto.request.LoginRequest;
import com.sigcbqr.model.dto.request.RegisterRequest;
import com.sigcbqr.model.dto.response.LoginResponse;
import com.sigcbqr.model.entity.Rol;
import com.sigcbqr.model.entity.Usuario;
import com.sigcbqr.repository.RolRepository;
import com.sigcbqr.repository.UsuarioRepository;
import com.sigcbqr.security.JwtTokenProvider;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
18
import java.time.LocalDateTime;
20
@Service
public class AuthService {
23
    private final AuthenticationManager authenticationManager;
    private final JwtTokenProvider tokenProvider;
    private final UsuarioRepository usuarioRepository;
    private final RolRepository rolRepository;
    private final PasswordEncoder passwordEncoder;
29
    public AuthService(AuthenticationManager authenticationManager,
    JwtTokenProvider tokenProvider,
    UsuarioRepository usuarioRepository,
    RolRepository rolRepository,
    PasswordEncoder passwordEncoder) {
35
        this.authenticationManager = authenticationManager;
        this.tokenProvider = tokenProvider;
        this.usuarioRepository = usuarioRepository;
        this.rolRepository = rolRepository;
        this.passwordEncoder = passwordEncoder;
40
    }
41
    public LoginResponse login(LoginRequest request) {
42
        authenticationManager.authenticate(
43
            new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
44
        );
45
        Usuario usuario = usuarioRepository.findByEmail(request.getEmail())
46
            .orElseThrow(() -> new ResourceNotFoundException("Usuario no encontrado"));
47
        String token = tokenProvider.generateToken(usuario.getId(), usuario.getEmail(),
48
            usuario.getRol().getNombre());
49
        return LoginResponse.builder()
50
            .id(usuario.getId())
51
            .nombre(usuario.getNombre())
52
            .email(usuario.getEmail())
53
            .rol(usuario.getRol().getNombre())
54
            .mensaje("Inicio de sesión exitoso")
55
            .build();
56
    }
57
    @Transactional
58
    public LoginResponse register(RegisterRequest request) {
59
        if (usuarioRepository.existsByEmail(request.getEmail())) {
60
            throw new BadRequestException("El correo ya está registrado");
61
        }
62
        Rol rol;
63
        if (request.getRolId() != null) {
64
            rol = rolRepository.findById(request.getRolId())
65
                .orElseThrow(() -> new ResourceNotFoundException("Rol no encontrado"));
66
        } else {
67
            rol = rolRepository.findByNombre("ESTUDIANTE")
68
                .orElseThrow(() -> new ResourceNotFoundException("Rol ESTUDIANTE no encontrado"));
69
        }
70
        Usuario usuario = Usuario.builder()
71
            .nombre(request.getNombre())
72
            .email(request.getEmail())
73
            .password(passwordEncoder.encode(request.getPassword()))
74
            .telefono(request.getTelefono())
75
    }

```

sigcb-qr-api\src\main\java\com\sigcbqr\service\AuthService.java (cont.)

```
82         .activo(true)
83         .rol(rol)
84         .build();
85
86     usuario = usuarioRepository.save(usuario);
87
88     String token = tokenProvider.generateToken(usuario.getId(), usuario.getEmail(),
89         usuario.getRol().getNombre());
90
91     return LoginResponse.builder()
92         .id(usuario.getId())
93         .nombre(usuario.getNombre())
94         .email(usuario.getEmail())
95         .rol(usuario.getRol().getNombre())
96         .mensaje("Registro exitoso")
97         .build();
98 }
99
100 public void logout(String token, LocalDateTime expiration) {
101     String jti = tokenProvider.getJtiFromToken(token);
102     tokenProvider.getJwtBlacklistService().blacklist(jti, expiration);
103 }
104
105 public Usuario getCurrentUser(Long userId) {
106     return usuarioRepository.findById(userId)
107         .orElseThrow(() -> new ResourceNotFoundException("Usuario", userId));
108 }
109 }
```


sigcb-qr-api\src\main\java\com\sigcbqr\controller\PrestamoController.java

```

package com.sigcbqr.controller;
2
import com.sigcbqr.model.dto.request.PrestamoRequest;
import com.sigcbqr.model.dto.response.ApiResponse;
import com.sigcbqr.model.dto.response.PageResponse;
import com.sigcbqr.model.dto.response.PrestamoResponse;
import com.sigcbqr.service.PrestamoService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;
16
@RestController
@RequestMapping("/api/prestamos")
@Tag(name = "Préstamos", description = "Gestión de préstamos, devoluciones y renovaciones")
public class PrestamoController {
21
    private final PrestamoService prestamoService;
23
    public PrestamoController(PrestamoService prestamoService) {
25        this.prestamoService = prestamoService;
26    }
27
    @GetMapping
    @Operation(summary = "Listar préstamos", description = "Obtiene todos los préstamos con paginación")
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    public ResponseEntity<PageResponse<PrestamoResponse>> listar(
32        @PageableDefault(size = 10, sort = "fechaPrestamo") Pageable pageable) {
33        var page = prestamoService.listar(pageable);
34        return ResponseEntity.ok(PageResponse.from(page));
35    }
36
    @GetMapping("/usuario/{usuarioId}")
    @Operation(summary = "Préstamos por usuario", description = "Obtiene los préstamos de un usuario específico")
    public ResponseEntity<PageResponse<PrestamoResponse>> listarPorUsuario(
40        @PathVariable Long usuarioId,
41        @PageableDefault(size = 10) Pageable pageable) {
42        var page = prestamoService.listarPorUsuario(usuarioId, pageable);
43        return ResponseEntity.ok(PageResponse.from(page));
44    }
45
    @GetMapping("/{id}")
    @Operation(summary = "Obtener préstamo", description = "Obtiene un préstamo por su ID")
    public ResponseEntity<ApiResponse> obtener(@PathVariable Long id) {
48        var prestamo = prestamoService.obtener(id);
49        return ResponseEntity.ok(ApiResponse.success("Préstamo encontrado", prestamo));
51    }
52
    @PostMapping
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    @Operation(summary = "Crear préstamo", description = "Registra un nuevo préstamo")
    public ResponseEntity<ApiResponse> crear(@Valid @RequestBody PrestamoRequest request) {
56        var prestamo = prestamoService.crear(request);
57        return ResponseEntity.ok(ApiResponse.created("Préstamo registrado", prestamo));
59    }
60
    @PutMapping("/{id}/devolver")
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    @Operation(summary = "Devolver libro", description = "Registra la devolución de un préstamo")
    public ResponseEntity<ApiResponse> devolver(@PathVariable Long id) {
65        var prestamo = prestamoService.devolver(id);
66        return ResponseEntity.ok(ApiResponse.success("Devolución registrada", prestamo));
67    }
68
    @PutMapping("/{id}/renovar")
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    @Operation(summary = "Renovar préstamo", description = "Renueva un préstamo activo")
    public ResponseEntity<ApiResponse> renovar(@PathVariable Long id) {
72        var prestamo = prestamoService.renovar(id);
73        return ResponseEntity.ok(ApiResponse.success("Préstamo renovado", prestamo));
74    }
75 }
76

```

sigcb-qr-api\src\main\java\com\sigcbqr\service\PrestamoService.java

```

package com.sigcbqr.service;
2
import com.sigcbqr.exception.BadRequestException;
import com.sigcbqr.exception.ResourceNotFoundException;
import com.sigcbqr.model.dto.request.PrestamoRequest;
import com.sigcbqr.model.dto.response.PrestamoResponse;
import com.sigcbqr.model.entity.*;
import com.sigcbqr.repository.*;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
13
import java.time.LocalDateTime;
15
@Service
public class PrestamoService {
18
19     private final PrestamoRepository prestamoRepository;
20     private final UsuarioRepository usuarioRepository;
21     private final InventarioRepository inventarioRepository;
22     private final LibroRepository libroRepository;
23
24     public PrestamoService(PrestamoRepository prestamoRepository,
25                           UsuarioRepository usuarioRepository,
26                           InventarioRepository inventarioRepository,
27                           LibroRepository libroRepository) {
28         this.prestamoRepository = prestamoRepository;
29         this.usuarioRepository = usuarioRepository;
30         this.inventarioRepository = inventarioRepository;
31         this.libroRepository = libroRepository;
32     }
33
34     public Page<PrestamoResponse> listar(Pageable pageable) {
35         return prestamoRepository.findAll(pageable).map(this::toResponse);
36     }
37
38     public Page<PrestamoResponse> listarPorUsuario(Long usuarioId, Pageable pageable) {
39         return prestamoRepository.findByUsuarioIdOrderByFechaPrestamoDesc(usuarioId, pageable)
40             .map(this::toResponse);
41     }
42
43     public PrestamoResponse obtener(Long id) {
44         Prestamo prestamo = prestamoRepository.findById(id)
45             .orElseThrow(() -> new ResourceNotFoundException("Préstamo", id));
46         return toResponse(prestamo);
47     }
48
49     @Transactional
50     public PrestamoResponse crear(PrestamoRequest request) {
51         Usuario usuario = usuarioRepository.findById(request.getUsuarioId())
52             .orElseThrow(() -> new ResourceNotFoundException("Usuario", request.getUsuarioId()));
53
54         Inventario inventario = inventarioRepository.findById(request.getInventarioId())
55             .orElseThrow(() -> new ResourceNotFoundException("Ejemplar", request.getInventarioId()));
56
57         if (!"DISPONIBLE".equals(inventario.getEstado())) {
58             throw new BadRequestException("El ejemplar no está disponible");
59         }
60
61         long prestamosActivos = prestamoRepository.countByUsuarioIdAndEstado(request.getUsuarioId(), "ACTIVO");
62         if (prestamosActivos >= 5) {
63             throw new BadRequestException("El usuario tiene demasiados préstamos activos");
64         }
65
66         Prestamo prestamo = Prestamo.builder()
67             .usuario(usuario)
68             .inventario(inventario)
69             .fechaPrestamo(LocalDateTime.now())
70             .fechaVencimiento(LocalDateTime.now().plusDays(7))
71             .estado("ACTIVO")
72             .build();
73
74         inventario.setEstado("PRESTADO");
75         inventarioRepository.save(inventario);
76
77         Libro libro = inventario.getLibro();
78         libro.setEjemplaresDisponibles(libro.getEjemplaresDisponibles() - 1);
79         libroRepository.save(libro);
80
81         prestamo = prestamoRepository.save(prestamo);

```

sigcb-qr-api\src\main\java\com\sigcbqr\service\PrestamoService.java (cont.)

```

82     return toResponse(prestamo);
83 }
84
85 @Transactional
86 public PrestamoResponse devolver(Long id) {
87     Prestamo prestamo = prestamoRepository.findById(id)
88         .orElseThrow(() -> new ResourceNotFoundException("Préstamo", id));
89
90     if ("DEVUELTO".equals(prestamo.getEstado())) {
91         throw new BadRequestException("El préstamo ya fue devuelto");
92     }
93
94     prestamo.setFechaDevolucion(LocalDateDateTime.now());
95     prestamo.setEstado("DEVUELTO");
96
97     Inventario inventario = prestamo.getInventario();
98     inventario.setEstado("DISPONIBLE");
99     inventarioRepository.save(inventario);
100
101     Libro libro = inventario.getLibro();
102     libro.setEjemplaresDisponibles(libro.getEjemplaresDisponibles() + 1);
103     libroRepository.save(libro);
104
105     // Generar multa si está vencido
106     if (prestamo.getFechaVencimiento().isBefore(LocalDateDateTime.now())) {
107         // Multa lógica here - handled by MultaService
108     }
109
110     prestamo = prestamoRepository.save(prestamo);
111     return toResponse(prestamo);
112 }
113
114 @Transactional
115 public PrestamoResponse renovar(Long id) {
116     Prestamo prestamo = prestamoRepository.findById(id)
117         .orElseThrow(() -> new ResourceNotFoundException("Préstamo", id));
118
119     if (!"ACTIVO".equals(prestamo.getEstado())) {
120         throw new BadRequestException("Solo se pueden renovar préstamos activos");
121     }
122
123     prestamo.setEstado("RENOVADO");
124     prestamo.setObservaciones("Renovado - nueva fecha: " + LocalDateDateTime.now().plusDays(7));
125     prestamoRepository.save(prestamo);
126
127     Prestamo nuevoPrestamo = Prestamo.builder()
128         .usuario(prestamo.getUsuario())
129         .inventario(prestamo.getInventario())
130         .fechaPrestamo(LocalDateDateTime.now())
131         .fechaVencimiento(LocalDateDateTime.now().plusDays(7))
132         .estado("ACTIVO")
133         .observaciones("Renovación del préstamo #" + prestamo.getId())
134         .build();
135
136     nuevoPrestamo = prestamoRepository.save(nuevoPrestamo);
137     return toResponse(nuevoPrestamo);
138 }
139
140 private PrestamoResponse toResponse(Prestamo prestamo) {
141     return PrestamoResponse.builder()
142         .id(prestamo.getId())
143         .usuarioNombre(prestamo.getUsuario().getNombre())
144         .libroTitulo(prestamo.getInventario().getLibro().getTitulo())
145         .codigoEjemplar(prestamo.getInventario().getCodigoEjemplar())
146         .fechaPrestamo(prestamo.getFechaPrestamo())
147         .fechaVencimiento(prestamo.getFechaVencimiento())
148         .fechaDevolucion(prestamo.getFechaDevolucion())
149         .estado(prestamo.getEstado())
150         .observaciones(prestamo.getObservaciones())
151         .build();
152 }
153 }

```

sigcb-qr-api\src\main\java\com\sigcbqr\config\CacheConfig.java

```

package com.sigcbqr.config;
2
import org.springframework.beans.factory.annotation.Value;
import org.springframework.cache.CacheManager;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.cache.RedisCacheManager;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext;
13
import java.time.Duration;
15
@Configuration
@EnableCaching
public class CacheConfig {
19
20     @Value("${app.cache.default-ttl-seconds}")
21     private int defaultTtlSeconds;
22
23     @Bean
24     public CacheManager cacheManager(RedisConnectionFactory connectionFactory) {
25         RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
26             .entryTtl(Duration.ofSeconds(defaultTtlSeconds))
27             .disableCachingNullValues()
28             .serializeValuesWith(
29                 RedisSerializationContext.SerializationPair.fromSerializer(
30                     new GenericJackson2JsonRedisSerializer());
31
32         return RedisCacheManager.builder(connectionFactory)
33             .cacheDefaults(config)
34             .build();
35     }
36

```

sigcb-qr-api\src\main\java\com\sigcbqr\exception\GlobalExceptionHandler.java

```

package com.sigcbqr.exception;
2
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
10
import java.net.URI;
import java.util.HashMap;
import java.util.Map;
14
@RestControllerAdvice
public class GlobalExceptionHandler {
17
18     public static final String BASE_ERROR_URL = "https://api.sigcbqr.com/errors";
19
20     @ExceptionHandler(ResourceNotFoundException.class)
21     public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
22         ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
23         pd.setType(URI.create(BASE_ERROR_URL + "/not-found"));
24         pd.setTitle("Recurso no encontrado");
25         pd.setProperty("timestamp", System.currentTimeMillis());
26         return pd;
27     }
28
29     @ExceptionHandler(BadRequestException.class)
30     public ProblemDetail handleBadRequest(BadRequestException ex) {
31         ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, ex.getMessage());
32         pd.setType(URI.create(BASE_ERROR_URL + "/bad-request"));
33         pd.setTitle("Solicitud inválida");
34         pd.setProperty("timestamp", System.currentTimeMillis());
35         return pd;
36     }
37
38     @ExceptionHandler(BadCredentialsException.class)
39     public ProblemDetail handleBadCredentials(BadCredentialsException ex) {
40         ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, "Credenciales inválidas");
41         pd.setType(URI.create(BASE_ERROR_URL + "/unauthorized"));
42         pd.setTitle("No autorizado");
43         pd.setProperty("timestamp", System.currentTimeMillis());
44         return pd;
45     }
46
47     @ExceptionHandler(MethodArgumentNotValidException.class)
48     public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
49         Map<String, String> errors = new HashMap<>();
50         for (FieldError error : ex.getBindingResult().getFieldErrors()) {
51             errors.put(error.getField(), error.getDefaultMessage());
52         }
53         ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Error de validación");
54         pd.setType(URI.create(BASE_ERROR_URL + "/validation"));
55         pd.setTitle("Error de validación");
56         pd.setProperty("errors", errors);
57         pd.setProperty("timestamp", System.currentTimeMillis());
58         return pd;
59     }
60
61     @ExceptionHandler(Exception.class)
62     public ProblemDetail handleGeneral(Exception ex) {
63         ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR,
64             "Error interno del servidor: " + ex.getMessage());
65         pd.setType(URI.create(BASE_ERROR_URL + "/internal-error"));
66         pd.setTitle("Error interno");
67         pd.setProperty("timestamp", System.currentTimeMillis());
68         return pd;
69     }
70 }

```

sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Prestamo.java

```

package com.sigcbqr.model.entity;
2
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;
6
import java.time.LocalDateTime;
8
@Entity
@Table(name = "prestamos")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
public class Prestamo {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @ManyToOne(fetch = FetchType.LAZY)
20     @JoinColumn(name = "usuario_id", nullable = false)
21     private Usuario usuario;
22
23     @ManyToOne(fetch = FetchType.LAZY)
24     @JoinColumn(name = "inventario_id", nullable = false)
25     private Inventario inventario;
26
27     @Column(name = "fecha_prestamo")
28     private LocalDateTime fechaPrestamo;
29
30     @Column(name = "fecha_vencimiento", nullable = false)
31     private LocalDateTime fechaVencimiento;
32
33     @Column(name = "fecha_devolucion")
34     private LocalDateTime fechaDevolucion;
35
36     @Column(length = 30)
37     private String estado;
38
39     @Column(columnDefinition = "TEXT")
40     private String observaciones;
41
42     @Column(name = "created_at")
43     private LocalDateTime createdAt;
44
45     @Column(name = "updated_at")
46     private LocalDateTime updatedAt;
47
48     @PrePersist
49     protected void onCreate() {
50         createdAt = LocalDateTime.now();
51         updatedAt = LocalDateTime.now();
52         if (estado == null) estado = "ACTIVO";
53         if (fechaPrestamo == null) fechaPrestamo = LocalDateTime.now();
54     }
55
56     @PreUpdate
57     protected void onUpdate() {
58         updatedAt = LocalDateTime.now();
59     }
60

```

sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Libro.java

```

package com.sigcbqr.model.entity;
2
import jakarta.persistence.*;
import lombok.*;
5
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;
9
@Entity
@Table(name = "libros")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Libro {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @Column(nullable = false, length = 300)
20     private String titulo;
21
22     @Column(unique = true, length = 20)
23     private String isbn;
24
25     @Column(name = "anio_publicacion")
26     private Integer anioPublicacion;
27
28     @Column(length = 50)
29     private String edicion;
30
31     @Column(name = "ejemplares_totales")
32     private Integer ejemplaresTotales;
33
34     @Column(name = "ejemplares_disponibles")
35     private Integer ejemplaresDisponibles;
36
37     @Column(length = 100)
38     private String ubicacion;
39
40     @Column(columnDefinition = "TEXT")
41     private String descripcion;
42
43     private String portada;
44
45     private Boolean activo;
46
47     @ManyToOne(fetch = FetchType.LAZY)
48     @JoinColumn(name = "categoria_id")
49     private Categoria categoria;
50
51     @ManyToOne(fetch = FetchType.LAZY)
52     @JoinColumn(name = "editorial_id")
53     private Editorial editorial;
54
55     @ManyToMany(fetch = FetchType.LAZY)
56     @JoinTable(
57         name = "libro_autores",
58         joinColumns = @JoinColumn(name = "libro_id"),
59         inverseJoinColumns = @JoinColumn(name = "autor_id")
60     )
61     @Builder.Default
62     private Set<Autor> autores = new HashSet<>();
63
64     @Column(name = "created_at")
65     private LocalDateTime createdAt;
66
67     @Column(name = "updated_at")
68     private LocalDateTime updatedAt;
69
70     @PrePersist
71     protected void onCreate() {
72         createdAt = LocalDateTime.now();
73         updatedAt = LocalDateTime.now();
74         if (activo == null) activo = true;
75         if (ejemplaresTotales == null) ejemplaresTotales = 1;
76         if (ejemplaresDisponibles == null) ejemplaresDisponibles = ejemplaresTotales;
77     }
78
79     @PreUpdate
80     protected void onUpdate() {
81         updatedAt = LocalDateTime.now();

```

sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Libro.java (cont.)

```
82 }
```

```
83
```


sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Usuario.java

```

package com.sigcbqr.model.entity;
2
import jakarta.persistence.*;
import lombok.*;
5
import java.time.LocalDateTime;
7
@Entity
@Table(name = "usuarios")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Usuario {
12
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @Column(nullable = false, length = 100)
18     private String nombre;
19
20     @Column(nullable = false, unique = true, length = 150)
21     private String email;
22
23     @Column(nullable = false)
24     private String password;
25
26     @Column(length = 20)
27     private String telefono;
28
29     private String foto;
30
31     @Column(nullable = false)
32     private Boolean activo;
33
34     @ManyToOne(fetch = FetchType.EAGER)
35     @JoinColumn(name = "rol_id", nullable = false)
36     private Rol rol;
37
38     @Column(name = "created_at")
39     private LocalDateTime createdAt;
40
41     @Column(name = "updated_at")
42     private LocalDateTime updatedAt;
43
44     @PrePersist
45     protected void onCreate() {
46         createdAt = LocalDateTime.now();
47         updatedAt = LocalDateTime.now();
48         if (activo == null) activo = true;
49     }
50
51     @PreUpdate
52     protected void onUpdate() {
53         updatedAt = LocalDateTime.now();
54     }
55 }

```

db\procs\sp_crear_prestamo.sql

```

CREATE OR REPLACE PROCEDURE sp_crear_prestamo(
  2  IN p_usuario_id BIGINT,
  3  IN p_inventario_id BIGINT,
  4  IN p_dias_prestamo INT DEFAULT 7,
  5  INOUT p_prestamo_id BIGINT DEFAULT NULL
)6
7  LANGUAGE plpgsql
AS $$
DECLARE
10  v_estado_inventario VARCHAR;
11  v_prestamos_activos BIGINT;
BEGIN
13  SELECT estado INTO v_estado_inventario FROM inventario WHERE id = p_inventario_id;
14  IF v_estado_inventario IS NULL THEN
15      RAISE EXCEPTION 'Ejemplar no encontrado';
16  END IF;
17  IF v_estado_inventario != 'DISPONIBLE' THEN
18      RAISE EXCEPTION 'El ejemplar no está disponible';
19  END IF;
20
21  SELECT COUNT(*) INTO v_prestamos_activos
22  FROM prestamos WHERE usuario_id = p_usuario_id AND estado = 'ACTIVO';
23  IF v_prestamos_activos >= 5 THEN
24      RAISE EXCEPTION 'El usuario tiene demasiados préstamos activos';
25  END IF;
26
27  INSERT INTO prestamos (usuario_id, inventario_id, fecha_prestamo, fecha_vencimiento, estado)
28  VALUES (p_usuario_id, p_inventario_id, NOW(), NOW() + (p_dias_prestamo || ' days')::INTERVAL, 'ACTIVO')
29  RETURNING id INTO p_prestamo_id;
30
31  UPDATE inventario SET estado = 'PRESTADO' WHERE id = p_inventario_id;
32
33  UPDATE libros l
34  SET ejemplares_disponibles = ejemplares_disponibles - 1
35  FROM inventario i
36  WHERE i.id = p_inventario_id AND l.id = i.libro_id;
END;
$$;

```

db\procs\sp_devolver_prestamo.sql

```

CREATE OR REPLACE PROCEDURE sp_devolver_prestamo(
  2  IN p_prestamo_id BIGINT,
  3  INOUT p_multa_generada BOOLEAN DEFAULT FALSE
)4
  5  LANGUAGE plpgsql
AS $$
DECLARE
  8  v_estado VARCHAR;
  9  v_inventario_id BIGINT;
10  v_libro_id BIGINT;
11  v_fecha_vencimiento TIMESTAMP;
12  v_dias_retraso INT;
BEGIN
14  SELECT estado, inventario_id, fecha_vencimiento
15  INTO v_estado, v_inventario_id, v_fecha_vencimiento
16  FROM prestamos WHERE id = p_prestamo_id;
17
18  IF v_estado IS NULL THEN
19    RAISE EXCEPTION 'Préstamo no encontrado';
20  END IF;
21  IF v_estado = 'DEVUELTO' THEN
22    RAISE EXCEPTION 'El préstamo ya fue devuelto';
23  END IF;
24
25  UPDATE prestamos
26  SET estado = 'DEVUELTO', fecha_devolucion = NOW()
27  WHERE id = p_prestamo_id;
28
29  UPDATE inventario SET estado = 'DISPONIBLE' WHERE id = v_inventario_id;
30
31  UPDATE libros l
32  SET ejemplares_disponibles = ejemplares_disponibles + 1
33  FROM inventario i
34  WHERE i.id = v_inventario_id AND l.id = i.libro_id;
35
36  IF v_fecha_vencimiento < NOW() THEN
37    v_dias_retraso := EXTRACT(DAY FROM (NOW() - v_fecha_vencimiento));
38    INSERT INTO multas (prestamo_id, usuario_id, monto, pagada, fecha_multa)
39    SELECT p_prestamo_id, p.usuario_id, v_dias_retraso * 0.50, false, NOW()
40    FROM prestamos p WHERE p.id = p_prestamo_id;
41    p_multa_generada := TRUE;
42  END IF;
END;
$$;

```

db\procs\sp_reporte_prestamos_diarios.sql

```
CREATE OR REPLACE PROCEDURE sp_reporte_prestamos_diarios(  
  2  IN p_fecha DATE,  
  3  INOUT p_ref REFCURSOR  
  4)  
  5  LANGUAGE plpgsql  
AS $$  
BEGIN  
  8  OPEN p_ref FOR  
  9      SELECT p.id, p.fecha_prestamo, p.estado,  
10          u.nombre AS usuario_nombre,  
11          l.titulo AS libro_titulo  
12  FROM prestamos p  
13  JOIN usuarios u ON u.id = p.usuario_id  
14  JOIN inventario i ON i.id = p.inventario_id  
15  JOIN libros l ON l.id = i.libro_id  
16  WHERE p.fecha_prestamo::DATE = p_fecha  
17  ORDER BY p.fecha_prestamo;  
END;  
$$;
```